



INTRODUCTION À LA SÉCURITÉ INFORMATIQUE - 3E ANNÉE (INGÉNIEURS)

Université 8 Mai 1945 - Département d'informatique , Guelma

Préparé par

- Bagha Tasnim
- Merah Khaoula
- Djerrab Sarah
- Kolli adem

RAPPORT DE PROJET

Mai 2025

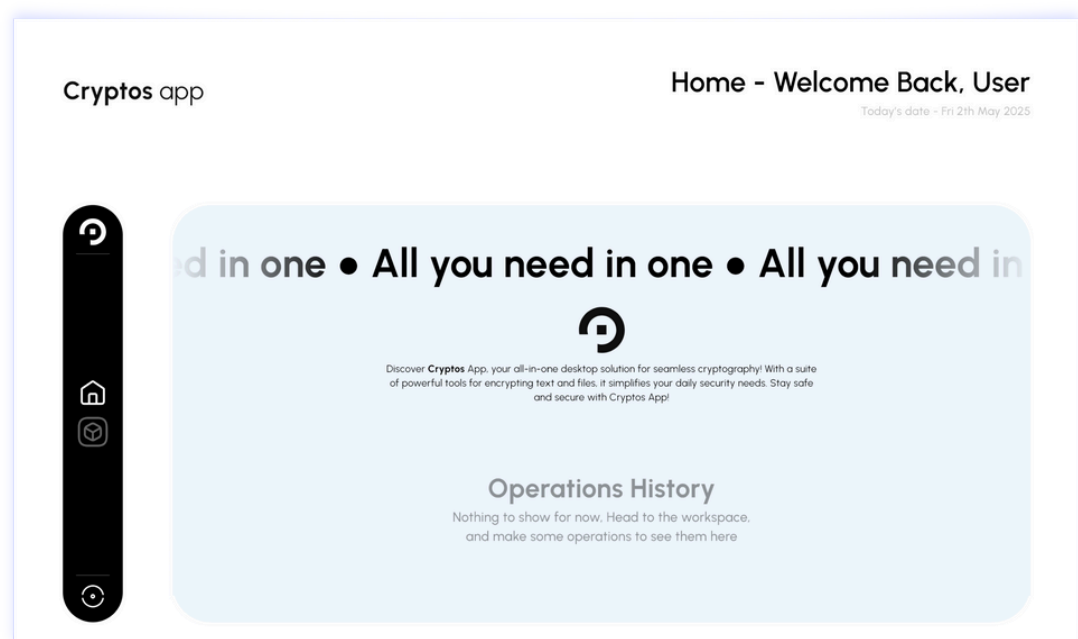
Implémentation des méthodes de cryptographie classique et moderne

 **Cryptos app**



TABLE DES MATIÈRES

	Début	Fin
Introduction	Page 01	Page 02
Documentation	Page 03	Page 10
Rapport Technique	Page 11	Page 15
Livrables	Page 15	Page 15
Conclusion	Page 16	Page 16



1. INTRODUCTION

1.1 Aperçu de la Cryptographie

La cryptographie joue un rôle fondamental dans la sécurité informatique moderne, en assurant la **confidentialité**, l'**intégrité** et l'**authenticité**, l'**authentification** et la **non-répudiation** des données. Elle repose sur des méthodes mathématiques pour **transformer des informations lisibles en données chiffrées**, rendant leur accès impossible aux parties non autorisées.

Les méthodes de cryptographie sont divisées en deux catégories principales : la **cryptographie classique**, qui repose sur des techniques historiques, et la **cryptographie moderne**, qui utilise des algorithmes avancés adaptés à l'ère numérique.

1.2 Objectifs du Projet

L'objectif principal de ce projet est de concevoir une application qui permet d'explorer et de mettre en œuvre des algorithmes de cryptographie classique et moderne. Grâce à ce projet, nous visons à :

- Approfondir nos connaissances des différentes techniques de chiffrement.
- Comprendre les différences entre les approches classiques et modernes.
- Développer une interface utilisateur intuitive pour tester et valider les algorithmes implémentés.

1.3 Portée du Projet

Après avoir pris en compte les exigences du projet et les compétences de notre équipe, nous avons sélectionné les algorithmes de cryptographie suivants pour l'implémentation :

Cryptographie classique :

1. Chiffrement par substitution
 - o Chiffrement de Vigenère : un chiffrement par substitution polyalphabétique utilisant un mot-clé pour déterminer plusieurs décalages
 - o Chiffrement de Playfair : un autre type de chiffrement par substitution qui utilise une grille 5x5 pour convertir des paires de lettres
2. Chiffrement par transposition (permutation)
 - o Transposition complexe par colonne : un chiffrement par transposition qui réorganise le texte clair en lignes, puis le lit en colonnes selon une clé

- Transposition carrée polybique : une forme de chiffrement de transposition où les caractères sont réarrangés en une grille carrée

Cryptographie moderne :

1. Chiffrement à clé secrète

- Algorithme DES (Data Encryption Standard) : un chiffrement symétrique par blocs qui a été largement utilisé mais qui est désormais considéré comme obsolète
- Algorithme AES (Advanced Encryption Standard) : un chiffrement symétrique par blocs largement utilisé pour la transmission sécurisée de données

2. Chiffrement à clé publique

- Algorithme RSA (Rivest-Shamir-Adleman) : un système de cryptographie asymétrique utilisé pour la transmission sécurisée et les signatures numériques
- Algorithme Diffie-Hellman (partage de clés) : un protocole permettant de sécuriser le partage de clés cryptographiques sur un canal public

3. Hachage

- SHA-256 : une fonction de hachage cryptographique générant une valeur de hachage de 256 bits
- MD5 : une autre fonction de hachage cryptographique, bien que moins sécurisée que SHA-256

Ces choix offrent un échantillon représentatif des techniques classiques et contemporaines, ce qui nous permet de mettre en évidence l'évolution significative des méthodes cryptographiques.

1.4 Composition de l'équipe et répartition des tâches

Notre équipe de projet est composée de quatre membres, chacun étant responsable de parties spécifiques de l'implémentation :

- **Bagha Tasnim** : rédaction du rapport du projet, implémentation de l'algorithme AES, de la fonction de hachage MD5, et du chiffrement de Playfair
- **Djerrab Sarah** : rédaction du rapport du projet, conception de l'interface utilisateur sur Figma, implémentation de l'algorithme Diffie-Hellman, de la transposition carrée de Polybe, et de la fonction de hachage SHA-256
- **Merah Khaoula** : développement de l'interface en Flutter et intégration avec les scripts de cryptographie, implémentation de l'algorithme RSA et de la transposition complexe par colonne
- **Kolli Adem** : développement de l'interface en Flutter et intégration avec les scripts, conception de l'interface sur Figma, implémentation du chiffrement de Vigenère et de l'algorithme DES

2. DOCUMENTATION

2.1 Cryptographie Classique : Implémentation et Documentation

- **Introduction Générale**

La cryptographie classique repose sur des techniques de chiffrement développées avant l'avènement des ordinateurs modernes. Ces méthodes visent à transformer un message clair en un texte chiffré en appliquant des transformations basées sur des clés. Dans cette section, nous implémentons les techniques suivantes :

- Chiffrement de Vigenère
- Chiffrement de Playfair
- Chiffrement par transposition complexe par colonne
- Chiffrement carré Polybique (Polybius Square)

- **Méthodes Implémentées**

1. Chiffrement de Vigenère

Introduction

Le chiffre de Vigenère est un algorithme de substitution polyalphabétique qui utilise une clé répétitive pour déterminer le décalage de chaque lettre. Il offre une meilleure sécurité en réduisant les motifs communs observés dans les textes chiffrés.

Fonctionnalités

- vigenere_encrypt(plain_text, key):
 - Chiffre un texte clair en utilisant une clé.
 - Garde les caractères non alphabétiques inchangés.
- vigenere_decrypt(encrypted_text, key):
 - Déchiffre un texte chiffré en utilisant la clé correspondante.

Algorithme

1. Convertir la clé en majuscules pour uniformité.
2. Associer chaque caractère du texte avec une lettre de la clé (répétée si nécessaire).
3. Calculer les lettres chiffrées ou déchiffrées en fonction de leur position dans l'alphabet.
4. Laisser les caractères non alphabétiques inchangés.

2. Chiffrement de Playfair

Introduction

Le chiffre de Playfair utilise une matrice 5x5 pour chiffrer des paires de lettres. Il a été le premier chiffre par substitution digraphique, offrant une amélioration en termes de sécurité par rapport aux chiffres monoalphabétiques.

Fonctionnalités

Prétraitement du texte :

- Remplacement des caractères non alphabétiques.
- Ajout de lettres de remplissage (X) si nécessaire.

- Chiffrement :

- Substitution basée sur les positions des lettres dans la matrice.

- Déchiffrement :

- Processus inverse pour retrouver le texte clair.

Algorithme

1. Construire une matrice 5x5 à partir d'un mot-clé.
2. Diviser le texte en paires de lettres, en ajoutant X si nécessaire.
3. Appliquer les règles de chiffrement/déchiffrement :
 - Même ligne : décaler à droite/gauche.
 - Même colonne : décaler en bas/haut.
 - Différentes lignes et colonnes : prendre les lettres aux intersections.

3. Chiffrement par Transposition Complexe par Colonne

Introduction

Ce chiffre réarrange les caractères du texte clair en fonction d'une clé, en écrivant le texte dans une grille de colonnes. Les colonnes sont ensuite réarrangées selon l'ordre alphabétique des lettres de la clé.

Fonctionnalités

- `columnar_transposition_encrypt(plaintext, key):`

- Chiffre le texte clair en le réarrangeant dans une grille.
- Gère le remplissage automatique en ajoutant des caractères X si nécessaire.

- `columnar_transposition_decrypt(ciphertext, key):`

- Déchiffre le texte en inversant le processus de transposition.

Algorithme

1. Écrire le texte clair dans une grille avec un nombre de colonnes égal à la longueur de la clé.
2. Réarranger les colonnes selon l'ordre alphabétique des lettres de la clé.
3. Lire les colonnes dans l'ordre pour obtenir le texte chiffré.

4. Chiffrement Carré Polybique (Polybius Square)

Introduction

Le carré Polybique encode les lettres en paires de chiffres représentant leurs positions dans une matrice 5x5. Ce chiffre est simple mais efficace pour illustrer les principes de substitution.

Fonctionnalités

- create_array(key, omit_letter):
 - Génère une matrice 5x5 à partir d'un mot-clé.
- encode(sentence):
 - Transforme le texte clair en coordonnées numériques.
- decode(message):
 - Transforme les coordonnées en texte clair.
- valide_numbers(message):
 - Valide le format des coordonnées encodées.

Algorithme

1. Construire une matrice 5x5 en excluant une lettre (par défaut W).
2. Remplacer les lettres par leurs coordonnées (ligne, colonne).
3. Convertir les coordonnées en texte clair pour le déchiffrement.

• Résumé

Ces méthodes de cryptographie classique illustrent les concepts fondamentaux de substitution et de transposition. Elles permettent de mieux comprendre les bases des techniques modernes et leur importance historique dans la sécurité des communications.

2.2 Cryptographie Moderne : Implémentation et Documentation

• Introduction Générale

La cryptographie moderne repose sur des algorithmes complexes, souvent basés sur des mathématiques avancées, pour garantir la confidentialité et l'intégrité des données. Dans cette section, nous couvrons les implémentations suivantes :

- Chiffrement DES-like
- Chiffrement AES-128, AES-192, et AES-256
- Algorithme RSA
- Échange de clés Diffie-Hellman

Ces méthodes illustrent les bases de la cryptographie symétrique et asymétrique, tout en offrant une introduction aux standards modernes.

• Méthodes Implémentées

1. Chiffrement DES-like

Introduction

Inspiré du chiffrement DES, cette implémentation éducative utilise un réseau de Feistel simplifié pour chiffrer et déchiffrer des données binaires. Bien qu'il ne soit pas sécurisé pour une utilisation réelle, il aide à comprendre des concepts tels que la permutation, l'expansion, les S-Box et le mélange de clés.

Fonctionnalités

- permute(block, table): Permute les bits selon une table donnée.
- xor(bits1, bits2): Effectue un XOR bit-à-bit.
- f_function(R, K): Transforme la moitié droite d'un bloc avec une clé.
- des_encrypt(plain_text, key_str): Chiffre un texte clair.
- des_decrypt(cipher_text, key_str): Déchiffre un texte chiffré.

Algorithme

1. Appliquer une permutation initiale (IP) au message clair.
2. Diviser le message en deux moitiés : gauche (L) et droite (R).
3. Effectuer 16 tours du réseau de Feistel :
 - Étendre R à 48 bits grâce à une table d'expansion.
 - Effectuer un XOR entre la clé ronde et le résultat de l'expansion.
 - Passer le résultat dans une S-Box simplifiée pour une substitution.
 - Appliquer une permutation finale et combiner avec L via un XOR.
 - Inverser L et R pour le tour suivant.
4. Combiner L et R et appliquer une permutation inverse (IP^{-1}) pour obtenir le texte chiffré.

2. Chiffrement AES-128, AES-192, et AES-256

Introduction

AES (Advanced Encryption Standard) est un algorithme de chiffrement symétrique très utilisé pour sécuriser les données. Les variantes AES-128, AES-192 et AES-256 diffèrent par la taille de la clé et le nombre de tours (rounds).

Fonctionnalités communes

- expand_key(): Génère les clés de tour.
- aes_encrypt_block(): Chiffre un bloc de 16 octets.
- aes_decrypt_block(): Déchiffre un bloc de 16 octets.
- pad() et unpad(): Gèrent le remplissage PKCS#7.
- Base64 Encoding/Decoding: Encode ou décode les données pour la transmission.

Algorithme

1. Diviser les données en blocs de 16 octets.
2. Appliquer un XOR initial entre le bloc et la clé principale.

3. Effectuer un nombre de tours dépendant de la variante AES :
 - AES-128 : 10 tours
 - AES-192 : 12 tours
 - AES-256 : 14 tours
4. À chaque tour :
 - SubBytes : Remplacer chaque octet via une table de substitution (S-Box).
 - ShiftRows : Effectuer un décalage circulaire des lignes.
 - MixColumns : Mélanger les colonnes via des opérations algébriques.
 - AddRoundKey : Effectuer un XOR entre l'état courant et la clé de tour.
5. Après le dernier tour, exclure l'étape MixColumns.
6. Combiner les blocs pour obtenir le texte chiffré final.

3. Algorithme RSA

Introduction

RSA est un algorithme de cryptographie asymétrique qui utilise une paire de clés publique et privée pour chiffrer et déchiffrer les données. Il repose sur la difficulté de factoriser de grands nombres premiers.

Fonctionnalités

- generate_keys() : Génère une paire de clés (publique et privée).
- encrypt(message, public_key) : Chiffre un message texte.
- decrypt(cipher_text, private_key) : Déchiffre un texte chiffré.

Algorithme

1. Génération des clés :
 - Choisir deux grands nombres premiers p et q .
 - Calculer $n = p * q$ et $\phi = (p - 1) * (q - 1)$.
 - Choisir un entier e tel que $1 < e < \phi$ et e soit premier avec ϕ .
 - Calculer d , l'inverse modulaire de e modulo ϕ .
 - La clé publique est (e, n) et la clé privée est (d, n) .
2. Chiffrement :
 - Transformer le message en entier m tel que $0 \leq m < n$.
 - Calculer le texte chiffré c tel que $c = m^e \bmod n$.
3. Déchiffrement :
 - Calculer le texte en clair m tel que $m = c^d \bmod n$.
 - Convertir m en texte lisible.

4. Échange de clés Diffie-Hellman

Introduction

Diffie-Hellman est un protocole de cryptographie asymétrique permettant à deux parties d'établir un secret partagé sans le transmettre directement. Il repose sur des calculs modulaires et des nombres premiers.

Fonctionnalités

- is_prime(num) : Vérifie si un nombre est premier.
- generate_prime() : Génère un nombre premier aléatoire.
- diffie_hellman() : Réalise l'échange de clés et calcule un secret partagé.

Algorithme

1. Choisir un nombre premier public p et une base publique g .
2. Chaque partie choisit une clé privée secrète : a pour Alice et b pour Bob.
3. Calculer les clés publiques :
 - Alice calcule $A = g^a \text{ mod } p$.
 - Bob calcule $B = g^b \text{ mod } p$.
4. Échanger les clés publiques entre Alice et Bob.
5. Calculer le secret partagé :
 - Alice calcule $S = B^a \text{ mod } p$.
 - Bob calcule $S = A^b \text{ mod } p$.
 - Les deux obtiennent le même secret partagé S .

• Résumé

Ces implémentations modernes montrent comment les concepts avancés de cryptographie sont utilisés pour sécuriser les communications et les données. Elles illustrent des principes fondamentaux de la cryptographie symétrique et asymétrique, ainsi que les techniques d'échange sécurisé de clés.

2.3 Hashing Algorithms: SHA-256 and MD5

• Introduction Générale

Les algorithmes de hachage sont des outils fondamentaux en cryptographie. Ils permettent de transformer une donnée d'entrée (texte, fichier, etc.) en une empreinte unique et fixe, souvent utilisée pour vérifier l'intégrité des données ou comme base des signatures numériques. Cette section présente deux implémentations de hachage :

- SHA-256 : l'algorithme SHA-256.
- MD5 : Un algorithme de hachage classique et largement utilisé.

1. SHA-256 Hashing Algorithm

Introduction

Cet algorithme simplifié transforme une chaîne d'entrée en un hachage hexadécimal de longueur fixe (256 bits).

Fonctionnalités

- string_to_binary_list(s): Convertit une chaîne en une liste de blocs binaires de 32 bits.
- append1(lst, max_val): Ajoute un bit '1' et remplit avec des zéros pour atteindre la longueur cible.
- append_size(lst, original_size): Ajoute la taille originale du message sous forme binaire (64 bits).
- split(lst): Divise la liste en blocs de 512 bits.
- initialize_constants(): Génère 64 constantes basées sur les racines cubiques des premiers nombres premiers.
- rotr(x, n): Effectue une rotation circulaire à droite.
- sigma0(x) / sigma1(x): Fonctions spécifiques à SHA pour générer un planning des messages.
- computation(blocks): Génère les plannings de messages (W) pour chaque bloc.
- apply_salt(input_text, salt, salt_position): Applique un sel (prefixe ou suffixe) pour diversifier le hachage.
- compute_hash(input_text): Fonction principale pour calculer l'empreinte finale de 256 bits.

Algorithme

1. Conversion et Préparation :
 - Convertir la chaîne en blocs binaires.
 - Ajouter un bit '1' et remplir avec des zéros.
 - Ajouter la taille originale du message.
2. Diviser en Blocs :
 - Fractionner le message en blocs de 512 bits.
3. Initialisation :
 - Générer les constantes et initialiser les valeurs de hachage (a à h).
4. Compression :
 - Traiter chaque bloc à travers 64 itérations en utilisant les plannings (W) et les constantes.
5. Finalisation :
 - Combiner les valeurs finales pour produire un hachage de 256 bits.

2. MD5 Hashing Algorithm

Introduction

MD5 (Message-Digest Algorithm 5) est un algorithme de hachage qui produit une empreinte de 128 bits à partir d'une donnée d'entrée. Bien qu'il soit vulnérable à des attaques cryptographiques, il reste largement utilisé pour vérifier l'intégrité des fichiers ou comme checksum.

Fonctionnalités

- update(data) : Met à jour l'objet de hachage avec les octets de données d'entrée.
- digest() : Retourne l'empreinte binaire brute.
- hexdigest() : Retourne l'empreinte sous forme hexadécimale.
- rotateLeft(x,n) : Effectue une rotation bit-à-bit vers la gauche.
- F(x,y,z), G(x,y,z), H(x,y,z), I(x,y,z) : Fonctions non linéaires utilisées dans l'algorithme.
- XX(func,a,b,c,d,x,s,ac) : Fonction de transformation principale pour les opérations MD5.

Algorithme

1. Initialisation :
 - Initialiser quatre variables de 32 bits (A, B, C, D) avec des constantes fixes.
2. Remplissage :
 - Ajouter un bit '1', des zéros et la taille du message pour atteindre un multiple de 512 bits.
3. Traitement :
 - Diviser le message en blocs de 512 bits.
 - Effectuer quatre séries de transformations non linéaires sur chaque bloc.
4. Finalisation :
 - Combiner les valeurs de A, B, C, et D pour produire un hachage de 128 bits.

• Conclusion

Ces implémentations permettent de comprendre les mécanismes internes des algorithmes de hachage. Bien que MD5 soit obsolète pour des usages cryptographiques, il reste utile pour des vérifications basiques. Le SHA-256 offre une approche pédagogique pour explorer les concepts fondamentaux des algorithmes de hachage.

3. RAPPORT TECHNIQUE

3.1 Outils et Technologies Utilisés

- **Langages de Programmation**

- 1. Python :**

- Implémentation de la logique des algorithmes de cryptographie (classiques et modernes).
 - Scripts indépendants pour chaque algorithme (ex : vigenere.py, aes.py).
 - Résultats renvoyés via stdout pour être capturés par l'interface Flutter.

- 2. Dart / Flutter :**

- Employé pour développer l'interface utilisateur (UI) sous forme d'application multiplateforme.
 - Flutter a été choisi pour sa flexibilité et sa capacité à créer des interfaces performantes et esthétiques, tout en permettant d'intégrer des scripts Python localement grâce à l'utilisation du package process_run.

- **Environnements de Développement**

- 1. VS Code :**

- IDE principal avec les extensions Pylance pour Python et Flutter Tools pour Dart.

- 2. GitHub :**

- Gestion de version, collaboration, et suivi des tâches via GitHub Projects.

- **Design et Prototypage**

- 1. Figma :**

- Conception des maquettes UI avec une approche minimaliste axée sur la clarté et la sécurité.

- **Gestion des Dépendances**

1. Python (pip) :

- `process_run` : interaction entre Flutter et Python.

2. Flutter (pubspec.yaml) :

- `process_run`, `provider`, et autres packages nécessaires à l'interface et à l'exécution.

3.2 Algorithmes Implémentés et Exemples de Tests

- **Cryptographie Classique**

1. Chiffrement de Vigenère

Description : Substitution polyalphabétique avec mot-clé.

Test :

- Texte clair : HELLO
- Clé : KEY
- Résultat attendu : RIJVS

2. Chiffrement de Playfair

Description : Substitution par paires avec grille 5x5.

Test :

- Texte clair : HELLO
- Clé : KEY
- Résultat attendu : DBNVM

3. Transposition Complexe par Colonne

Description : Texte organisé en lignes, puis lu en colonnes selon une clé.

Test :

- Texte clair : ATTACKATDAWN
- Clé : ZEBRAS
- Résultat attendu : CWTDTTAAKNAA

4. Transposition Carrée Polybique

Description : Réarrangement en matrice carrée.

Test :

- Texte clair : ATTACKATDAWN
- Clé : ZEBRAS
- Résultat attendu : 15 51 51 15 22 34 15 51 23 15 42

- **Cryptographie Moderne**

1. DES (Data Encryption Standard)

Description : Chiffrement symétrique par blocs (64 bits).

Test :

- Texte clair : Bonjour tout le monde
- Clé : 12345678 (8 octets)
- Résultat attendu : Chiffrement en base64 (ex : U2FsdGVkX1+M2kZy...)

2. AES (Advanced Encryption Standard)

Description : Chiffrement symétrique (128/192/256 bits).

Test :

- Texte clair : HELLO WORLD
- Clé : 1234567890ABCDEF (128 bits)
- Résultat attendu : Sh1O2u3ebAwxxMXomTRLsw==

3. RSA

Description : Cryptographie asymétrique utilisée pour les signatures et le chiffrement.

Test :

- Texte clair : SECRETMESSAGE
- Clés générées dynamiquement (exemple) : 7291,17113
- Résultat attendu (exemple) : 15561 16935 14103 2262 16935 9458 3302 16935 15561 15561 10835 13696 16935

4. Diffie-Hellman

Description : Protocole d'échange de clé sécurisé.

Test :

- Utilisateurs A et B échangent leur clé publique, génèrent une clé commune.
- Résultat attendu : Clé partagée identique pour les deux utilisateurs
- exemple : N=11 , P=7, x=5, y=2
- result: shared_key_x=10,shared_key_y=5, secret_key=1

• Fonctions de Hachage

1. SHA-256

Description : Fonction de hachage sécurisée à 256 bits.

Test :

- Texte : password123
- Résultat attendu :
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

2. MD5

Description : Fonction plus ancienne, moins sécurisée.

Test :

- Texte : password123
- Résultat attendu : 482c811da5d5b4bc6d497ffa98491e38

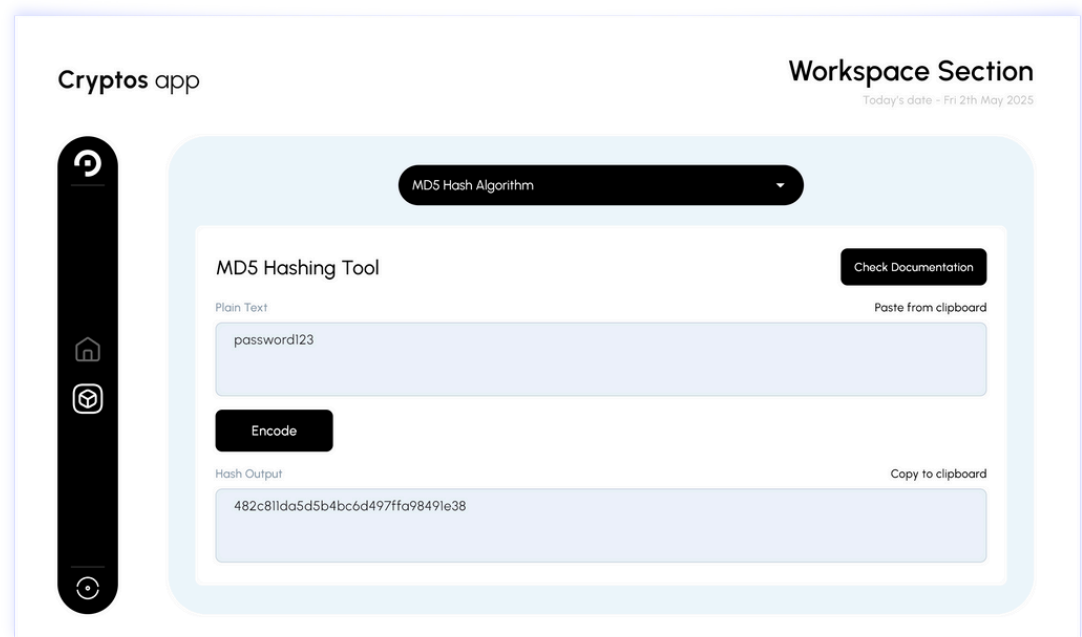
3.3 Interface Utilisateur

- **Design**

Palette: Bleu clair, blanc, noir (inspire sécurité et confiance)

Composants:

- Menu déroulant (pour AES, etc.)
- Zones de texte avec placeholders clairs
- Boutons d'action : Chiffrer, Déchiffrer



- **Fonctionnalités**

Documentation intégrée: Accès à une aide décrivant chaque algorithme, ses entrées et sorties.

Gestion des erreurs: Exemple : Clé RSA invalide – minimum 2048 bits requis

3.4 Défis et Solutions

- **Intégration Python-Flutter**

Problème: Pas de serveur intermédiaire

Solution: process_run permet d'exécuter localement les scripts Python depuis Flutter

- **Gestion des erreurs utilisateur**

Problème : Entrées incorrectes (ex : mauvaise taille de clé)

Solution : Validations Flutter + messages d'erreur précis

3.5 Validation et Résultats

- Chaque algorithme a été testé avec des jeux de données standards.
- Les résultats ont été comparés à des outils ou bibliothèques de référence (ex : <https://www.dcode.fr> , CyberChef).

- **Conclusion**

L'implémentation combine efficacité, lisibilité, modularité et pédagogie. Grâce à l'utilisation de Flutter pour l'interface et Python pour la logique algorithmique, le projet permet d'explorer concrètement les méthodes de chiffrement classiques et modernes, avec une expérience fluide et sécurisée, sans nécessiter de serveur distant.

4. LIVRABLES

Les éléments suivants seront fournis dans un lien Google Drive :

- Documentation complète du projet (rapport technique et manuel d'utilisation)
- Code source de l'application (scripts Python et interface Flutter)
- Fichier exécutable (.exe) permettant de lancer directement l'application sans installation préalable

Lien de téléchargement : \[à insérer]

5. CONCLUSION

La cryptographie est bien plus qu'un concept théorique : elle est au cœur de la cybersécurité moderne. Ce projet nous a permis de mieux comprendre son utilité réelle, son impact, et la rigueur nécessaire à sa mise en œuvre.

Nous avons développé une application de cryptographie complète, intégrant à la fois des algorithmes classiques (Vigenère, Playfair, transposition...) et modernes (AES, RSA, Diffie-Hellman...), ainsi que des fonctions de hachage (SHA-256, MD5).

L'interface utilisateur, conçue sous Flutter, offre une expérience intuitive pour tester ces méthodes.

Ce projet nous a ainsi permis de consolider nos connaissances en cryptographie, en programmation modulaire, en conception UI/UX, et en intégration de technologies hétérogènes. Il a également renforcé notre capacité à collaborer en équipe sur un projet complet.



 **Cryptos app**

MERCI !!

